DDL

CREATE

```
CREATE TABLE Estudiante (

   IdEstudiante     NUMBER(10),        -- Número entero de hasta 10 cifras
   Nombre           VARCHAR2(50) NOT NULL, -- Texto hasta 50 caracteres, obligatorio
   Edad             NUMBER(3),
   Email            VARCHAR2(100),
   FechaAlta        DATE,              -- Fecha completa (día, mes, año, hora)
   NotaMedia        NUMBER(4,2),       -- Número con decimales (ej: 8.75)
   Telefono         NUMBER(9),
   CodigoPostal     VARCHAR2(10),
   Activo           CHAR(1),           -- Un solo carácter (ej: 'S' o 'N')

   CONSTRAINT pk_estudiante PRIMARY KEY (IdEstudiante),

   CONSTRAINT uq_email UNIQUE (Email),

   CONSTRAINT ck_edad CHECK (Edad BETWEEN 0 AND 120),

   CONSTRAINT ck_email CHECK (
      Email IS NULL OR Email LIKE '%@%.%'
   ),

   CONSTRAINT ck_telefono CHECK (
      Telefono IS NULL OR Telefono BETWEEN 600000000 AND 999999999
   ),

   CONSTRAINT ck_cp CHECK (
      CodigoPostal IS NULL OR LENGTH(CodigoPostal) BETWEEN 4 AND 10
   ),

   CONSTRAINT ck_activo CHECK (
      Activo IN ('S', 'N')
   )
);
```

Tipo	Ejemplo	Para qué sirve
NUMBER(n)	NUMBER(10)	Número entero de hasta n cifras
NUMBER(n,m)	NUMBER(4,2)	Número con decimales (m decimales)
VARCHAR2(n)	VARCHAR2(50)	Texto de longitud variable
CHAR(n)	CHAR(1)	Texto de longitud fija
DATE	DATE	Fecha completa
CLOB	(no usado aquí)	Texto muy largo
BLOB	(no usado aquí)	Datos binarios

ALTER

```
ALTER TABLE Estudiante
ADD Telefono NUMBER(9);

ALTER TABLE Estudiante
MODIFY Nombre VARCHAR2(80);

ALTER TABLE Estudiante
ADD CONSTRAINT ck_email CHECK (Email LIKE '%@%.%');

ALTER TABLE Estudiante
DROP COLUMN Edad;

ALTER TABLE Estudiante
DROP CONSTRAINT ck_email;
```

DROP

```
DROP TABLE Estudiante;

DROP TABLE Estudiante CASCADE CONSTRAINTS; //MEJOR
```

DML

INSERT

```
INSERT INTO Estudiante (IdEstudiante, Nombre, Edad, Email)
VALUES (1, 'Laura', 20, 'laura@example.com');

INSERT INTO Estudiante
VALUES (2, 'Carlos', 22, 'carlos@example.com', DATE '2024-09-01');


INSERT ALL
  INTO Estudiante VALUES (3, 'Ana', 19, 'ana@example.com', DATE '2024-09-01')
  INTO Estudiante VALUES (4, 'Luis', 21, 'luis@example.com', DATE '2024-09-01')
SELECT * FROM dual;

INSERT INTO Estudiante (IdEstudiante, Nombre, Edad)
SELECT IdEstudiante + 100, Nombre, Edad
FROM Estudiante
WHERE Edad > 20;
```

UPDATE

```
UPDATE Estudiante
SET Edad = 21
WHERE IdEstudiante = 1;

UPDATE Estudiante
SET Nombre = 'Laura Gómez',
    Edad = 22
WHERE IdEstudiante = 1;

UPDATE Estudiante
SET Edad = (SELECT MAX(Edad) FROM Estudiante)
WHERE IdEstudiante = 2;
```

DELETE

```
DELETE FROM Estudiante
WHERE IdEstudiante = 4;
```

```
DELETE FROM Estudiante;

DELETE FROM Estudiante
WHERE Edad < (SELECT AVG(Edad) FROM Estudiante);
```

SELECT Y CONSULTAS

```
SELECT columnas → qué quieres ver
FROM tabla → de dónde
WHERE condición → filtrar
ORDER BY columna; → agrupar/ordenar resultados

SELECT Nombre, Edad
FROM Estudiante
WHERE Edad >= 18
ORDER BY Nombre;

SELECT * FROM  Estudiante; (from- de dónde)

SELECT Nombre, Email FROM Estudiante;

SELECT Nombre AS Alumno, Email AS Correo
FROM Estudiante;

SELECT * FROM Estudiante WHERE Edad > 20; (where-filtro)
SELECT * FROM Estudiante WHERE Edad = 18;
SELECT * FROM Estudiante WHERE Edad <>30;

SELECT * FROM Estudiante
WHERE Edad BETWEEN 18 AND 25;

SELECT * FROM Estudiante
WHERE CursoPeticion IN ('DAM', 'DAW');

SELECT * FROM Estudiante WHERE Nombre LIKE 'A%';
SELECT * FROM Estudiante WHERE Email LIKE '%@gmail.%';

SELECT * FROM Estudiante WHERE Telefono IS NULL;
SELECT * FROM Estudiante WHERE Email IS NOT NULL;

SELECT Nombre, Edad (order by-orden ascendente)
FROM Estudiante
ORDER BY Edad;
```

SELECT Nombre, Edad (orden descendente)
FROM Estudiante
ORDER BY Edad DESC;

SELECT Nombre, Edad
FROM Estudiante
ORDER BY Edad DESC, Nombre ASC;

SELECT DISTINCT CursoPeticion (eliminar duplicados)
FROM Estudiante;

SELECT CursoPeticion, COUNT(*) (group by-agrupar resultados)
FROM Estudiante
GROUP BY CursoPeticion;

SELECT CursoPeticion, COUNT(*)
FROM Estudiante
GROUP BY CursoPeticion
HAVING COUNT(*) > 1; (filtrar grupos (no filas))

FUNCIONES DE GRUPO			
COUNT(*)	SELECT COUNT(*) FROM Estudiante;	cuenta filas	filas = **5 estudiantes → 5**
AVG(columna)	SELECT AVG(Edad) FROM Estudiante;	media	edades = **10, 20, 30** → (10+20+30)/3 = **20**
SUM(columna)	SELECT SUM(Edad) FROM Estudiante;	suma	edades = **5, 7, 8** → 5+7+8 = **20**
MIN(columna)	SELECT MIN(Edad) FROM Estudiante;	mínimo	edades = **1, 2, 3, 4** → **1**
MAX(columna)	SELECT MAX(Edad) FROM Estudiante;	máximo	edades = **1, 2, 3, 4** → **4**
STDDEV	SELECT STDDEV(Edad) FROM Estudiante;	desviación típica	edades = **10, 10, 10** → **0** (no hay variación)
VARIANCE	SELECT	varianza	edades = **10, 10, 10**

	VARIANCE(Edad) FROM Estudiante;		→ **0**

FUNCIONES NUMÉRICAS			
ABS(n)	SELECT ABS(-5) FROM dual;	valor absoluto	`ABS(-5)` → 5
FLOOR(n)	SELECT FLOOR(95.9) FROM dual;	redondea hacia abajo	`FLOOR(95.9)` → 95
CEIL(n)	SELECT CEIL(95.1) FROM dual;	redondea hacia arriba	`CEIL(95.1)` → 96
MOD(a,b)	SELECT MOD(7,3) FROM dual;	resto de la división	`MOD(7,3)` → 1
POWER(a,b)	SELECT POWER(2,8) FROM dual;	potencia	`POWER(2,8)` → 256
TRUNC(n,dec)	SELECT TRUNC(1.856,2) FROM dual;	corta decimales	`TRUNC(1.856,2)` → 1.85
ROUND(n,dec)	SELECT ROUND(1.856,2) FROM dual;	redondea	`ROUND(1.856,2)` → 1.86
SIGN(n)	SELECT SIGN(-2) FROM dual;	signo	`SIGN(-2)` → -1

FUNCIONES DE LISTAS			
GREATEST(a,b,c)	SELECT GREATEST('HARRY', 'HARRIOT', 'HAROLD') FROM dual;	devuelve el mayor	HAROLD, HARRIOT, **HARRY** → **HARRY** es el mayor por orden alfabético
LEAST(a,b,c)	SELECT LEAST('HARRY', 'HARRIOT', 'HAROLD') FROM dual;	devuelve el menor	**HAROLD** es el menor alfabéticamente

FUNCIONES DE CADENA			
UPPER(cad)	SELECT UPPER('hola') FROM dual;	todo en mayúsculas	hola → HOLA
LOWER(cad)	SELECT LOWER('HOLA') FROM dual;	todo en minúsculas	HOLA → hola
INITCAP(cad)	SELECT INITCAP('mARIA') FROM dual;	primera letra mayúscula	mARIA → Maria
||...||	SELECT 'Hola ' || 'Mundo' FROM dual;	concatenación	Hola Mundo

FUNCIONES DE MANIPULACIÓN DE CADENA			
CONCAT(a,b)	SELECT CONCAT('Hola', 'Mundo') FROM dual;	Une dos cadenas	HolaMundo
CHR(n)	SELECT CHR(64) FROM dual;	carácter ASCII	**64 → '@'**
ASCII(c)	SELECT ASCII('A') FROM dual;	código ASCII	**'A' → 65**
LENGTH(cad)	SELECT LENGTH('Hola') FROM dual;	longitud	4
TRIM(cad)	SELECT TRIM(' hola ') FROM dual;	quita espacios	**"hola"**
LTRIM(cad)	SELECT LTRIM(' hola') FROM dual;	quita espacios a la izquierda	**"hola"**
RTRIM(cad)	SELECT RTRIM('hola ') FROM dual;	quita espacios a la derecha	**"hola"**

SUBSTR(cad, pos, n)	SELECT SUBSTR('Antonio', 2, 3) FROM dual;	subcadena	Posición 2 = **n** 3 caracteres → **nto**
REPLACE(cad, sub, nueva)	SELECT REPLACE('corazón','ó','o') FROM dual;	sustituye	corazón
TRANSLATE(cad, orig, dest)	SELECT TRANSLATE('hola','oa','01') FROM dual;	sustituye múltiples caracteres	o → 0 a → 1 Resultado: **h0l1**

FUNCIONES DE FECHA			
SYSDATE	SELECT SYSDATE FROM dual;	fecha actual	
ADD_MONTHS(fecha, n)	SELECT ADD_MONTHS(DATE '2024-01-01', 2) FROM dual;	suma meses	2024-01-01 + 2 meses → 2024-03-01
MONTHS_BETWEEN(f1,f2)	SELECT MONTHS_BETWEEN(DATE '2024-03-01', DATE '2024-01-01') FROM dual;	diferencia en meses	2
LAST_DAY(fecha)	SELECT LAST_DAY(DATE '2024-02-10') FROM dual;	último día del mes	2024-02-29
NEXT_DAY(fecha, 'MONDAY')	SELECT NEXT_DAY(DATE '2024-03-01','MONDAY') FROM dual;	próximo lunes después del 1 de marzo	

TRANSFORMACIÓN: TO_CHAR, TO_DATE, TO_NUMBER			
TO_CHAR	SELECT TO_CHAR(DATE '2024-03-05', 'dd/mm/yyyy') FROM dual;	convierte fecha o número a texto	**05/03/2024**
TO_DATE	SELECT TO_DATE('05/03/2024', 'dd/mm/yyyy') FROM dual;	convierte texto a fecha	
TO_NUMBER	SELECT TO_NUMBER('1234.56') FROM dual;	convierte texto a número	número **1234.56**

JOIN = unir filas de dos tablas usando una columna en común. Solo se combinan las filas que cumplen la condición del ON.

(Asignatura.CodCiclo = 1 → se une con CicloFormativo.CodCF = 1) SIN JOIN → CON JOIN

```
SELECT *
FROM Asignatura a, CicloFormativo c
WHERE a.CodCiclo = c.CodCiclo; (join antigüo, se unen todas las filas)
```

```
SELECT *
FROM Asignatura a
JOIN CicloFormativo c
ON a.CodCiclo = c.CodCiclo; (sólo se unen las filas que coinciden, suelen ser FK)
```

INNER JOIN → solo coincidencias

```
SELECT a.Nombre, c.Nombre
FROM Asignatura a
INNER JOIN CicloFormativo c
ON a.CodCiclo = c.CodCiclo;
```

LEFT JOIN → todo lo de la tabla izquierda + coincidencias

```
SELECT a.Nombre, c.Nombre
FROM Asignatura a
LEFT JOIN CicloFormativo c
ON a.CodCiclo = c.CodCiclo;
```

RIGHT JOIN → todo lo de la derecha + coincidencias

```
SELECT a.Nombre, c.Nombre
FROM Asignatura a
RIGHT JOIN CicloFormativo c
ON a.CodCiclo = c.CodCiclo;
```

FULL JOIN → todo de ambas tablas

```
SELECT a.Nombre, c.Nombre
FROM Asignatura a
FULL OUTER JOIN CicloFormativo c
ON a.CodCiclo = c.CodCiclo;
```

Las columnas no tienen que llamarse igual para poder igualarlas		
Tabla 1	**Tabla 2**	**JOIN**
CodDocente	F_CodDocente	ON CodDocente = F_CodDocente
CodCF	F_CodCiclo	ON CodCF = F_CodCiclo
IdAlumno	F_CodAlumno	ON IdAlumno = F_CodAlumno

EJEMPLO FINAL JOIN:

Tabla ASIGNATURA

CodAsig	**Nombre**	**CodCiclo**
1	BBDD	1
2	LDM	1
3	PROG	2

Tabla CICLO FORMATIVO

CodCiclo	**Nombre**
1	DAW
2	DAM

```
SELECT a.Nombre, c.Nombre
FROM Asignatura a
JOIN CicloFormativo c
ON a.CodCiclo = c.CodCiclo;
```

RESULTADO

Asignatura	**Ciclo**
BBDD	DAW
LDM	DAW
PROG	DAM

NATURAL JOIN → une todas las columnas que se llamen igual y no hace falta poner ON (no se suele usar)

```
SELECT *
FROM A
NATURAL JOIN B;
```

tabla A

CodCiclo	Nombre
1	BBDD
2	PROG

tabla B

CodCiclo	Ciclo
1	DAW
2	DAM

RESULTADO

CodCiclo	Nombre	Ciclo
1	BBDD	DAW
2	PROG	DAM

USING → solo funciona si las columnas se llaman igual en ambas tablas, sirve para no usar ON

```
SELECT *
FROM Asignatura
JOIN CicloFormativo
USING (CodCiclo);
```

SQL y SUBCONSULTAS

Ejemplo simple:

```
SELECT Nombre
FROM Alumno
WHERE CodAlum IN (
  SELECT CodAlumno
  FROM Matricula
);
```

La consulta interna:

```
SELECT CodAlumno FROM Matricula
```

(se ejecuta primero, y su resultado se usa en la consulta externa)

ANY y ALL

```
SELECT *
FROM Matricula
WHERE Nota > ANY (SELECT Nota FROM Matricula);
```

→ Devuelve las matrículas cuya nota es mayor que alguna de las notas de la subconsulta (todas menos la menor).

```
SELECT *
FROM Matricula
WHERE Nota > ALL (SELECT Nota FROM Matricula);
```
→ Devuelve las matrículas cuya nota es mayor que todas las notas de la subconsulta (la nota máxima).

SUBCONSULTA EN WHERE + IN

```
SELECT a.Nombre, m.Nota, al.Nombre, al.PrimerAp
FROM Matricula m
JOIN Asignatura a ON m.CodAsig = a.CodAsig
JOIN Alumno al ON m.CodAlumno = al.CodAlum
WHERE m.Nota < 5
AND al.CodAlum IN (
  SELECT CodAlum
  FROM Alumno
  WHERE PrimerAp IN ('Blanco', 'Muñoz')
);
```
Filtra las asignaturas suspendidas por alumnos Blanco o Muñoz.

SUBCONSULTA CON FUNCIÓN (AVG)

```
SELECT Nombre, PrimerAp, SegundoAp, DNI
FROM Alumno
WHERE CodAlum IN (
  SELECT CodAlumno
  FROM Matricula
  WHERE Nota > (SELECT AVG(Nota) FROM Matricula)
);
```
Devuelve los alumnos que tienen alguna nota superior a la media.

SUBCONSULTA CON JOIN DENTRO

```
SELECT *
FROM Alumno
WHERE CodAlum IN (
  SELECT m.CodAlumno
  FROM Matricula m
  JOIN Asignatura a ON m.CodAsig = a.CodAsig
  JOIN CicloFormativo c ON a.CodCF = c.CodCF
  WHERE c.Abreviatura = 'DAW'
);
```
Devuelve todos los datos de los alumnos matriculados en asignaturas del ciclo DAW

SUBCONSULTA CON EXISTS

(EXISTS devuelve TRUE si la subconsulta devuelve al menos una fila)

```
SELECT *
FROM Alumno al
WHERE EXISTS (
   SELECT 1
   FROM Matricula m
   JOIN Asignatura a ON m.CodAsig = a.CodAsig
   JOIN CicloFormativo c ON a.CodCF = c.CodCF
   WHERE m.CodAlumno = al.CodAlum
   AND c.Abreviatura = 'DAM'
);
```

Devuelve solo los alumnos que tienen matrículas en DAM.

SUBCONSULTAS CORRELACIONADAS

```
SELECT al.Nombre, al.PrimerAp, m.CodAsig, m.Nota
FROM Alumno al
JOIN Matricula m ON al.CodAlum = m.CodAlumno
WHERE m.Nota = (
   SELECT MAX(m2.Nota)
   FROM Matricula m2
   WHERE m2.CodAsig = m.CodAsig
);
```

Devuelve los alumnos que tienen la nota más alta en cada asignatura.

SUBCONSULTAS EN SELECT

```
SELECT a.Nombre AS ASIGNATURA,
     c.Nombre AS CICLO,
     (SELECT COUNT(*) FROM Matricula m WHERE m.CodAsig = a.CodAsig) AS
NUM_MATRICULADOS,
     (SELECT AVG(m2.Nota) FROM Matricula m2 WHERE m2.CodAsig = a.CodAsig) AS
MEDIA_NOTAS
FROM Asignatura a
JOIN CicloFormativo c ON a.CodCF = c.CodCF;
```

Devuelve una tabla con nombre de asignatura, nombre del ciclo, número de matriculados y la media de notas.

OPERADORES DE CONJUNTOS (UNION, MINUS, INTERSECT)

MINUS

```
SELECT Nombre
FROM Asignatura
WHERE CodCF = (SELECT CodCF FROM CicloFormativo WHERE Abreviatura = 'DAW')
MINUS
SELECT Nombre
FROM Asignatura
WHERE CodCF = (SELECT CodCF FROM CicloFormativo WHERE Abreviatura =
'DAM');
```

NOT EXISTS

```
SELECT *
FROM Alumno al
NOT EXISTS (
   SELECT 1
   FROM Matricula m
   WHERE m.CodAlumno = al.CodAlum
);
```

Devuelve los alumnos sin matrículas.

UNION

```
SELECT Nombre, PrimerAp, SegundoAp
FROM Alumno
WHERE CodAlum IN (
   SELECT CodAlumno
   FROM Matricula m
   JOIN Asignatura a ON m.CodAsig = a.CodAsig
   WHERE a.CodCF = 1
)
UNION
SELECT Nombre, PrimerAp, SegundoAp
FROM Alumno
WHERE CodAlum IN (
   SELECT CodAlumno
   FROM Matricula m
   JOIN Asignatura a ON m.CodAsig = a.CodAsig
   WHERE a.CodCF = 2
);
```

OPERADOR	QUÉ DEVUELVE	EJEMPLO	RESULTADO DEL EJEMPLO

UNION	Une los resultados de dos consultas **sin duplicados**.	SELECT Nombre FROM DAW UNION SELECT Nombre FROM DAM;	Todos los nombres que aparecen en DAW o en DAM, **sin repetir ninguno**.
UNION ALL	Igual que UNION, pero **con duplicados**.	SELECT Nombre FROM DAW UNION ALL SELECT Nombre FROM DAM;	Todos los nombres, **incluyendo los repetidos**.
MINUS	Devuelve lo que está en la **primera consulta** pero **NO** en la segunda.	SELECT Nombre FROM DAW MINUS SELECT Nombre FROM DAM;	Asignaturas que están en DAW **pero no** en DAM.
INTERSECT	Devuelve solo los valores que están en **ambas consultas**.	SELECT Nombre FROM DAW INTERSECT SELECT Nombre FROM DAM;	Asignaturas que están **tanto en DAW como en DAM**.